Silicon & Handset — Deep Forensic Code Audit: Logic Bugs, Broken Physics Incentives & Latent Architectural Time-Bombs

Target Audience: Autonomous Implementation Models (`GPT Sol 6.1`, `Gemini 4 Argon`, `Codex`, `Agy`)
Scope: Line-by-line static & mathematical verification of `SPINOFF_SOURCE/js/sim.js`, `SPINOFF_SOURCE/js/phone-engineering.js`, `SPINOFF_SOURCE/js/human-core.js`, `SPINOFF_SOURCE/js/ui.js`, and `SPINOFF_SOURCE/css/game.css`.
Purpose: Expose active logic defects, perverse mathematical incentives, silent desynchronizations, and latent scalability/determinism time-bombs that currently pass baseline tests (`22/22` unit, `4/4` browser) or appear benign today, **but either silently corrupt behavior right now or will cause catastrophic failures as `v0.3.0+` expands the simulation**.
Hard Constraints (`AGENTS.md`):

- `SPINOFF_SOURCE/js/human-core.js` (SHA-256 `fa83eaa0ef3407f26f21675161f632a4365242f67fea7b0c192e59e75b9d5c32`) and `SPINOFF_SOURCE/CORE_LOCK.json` are **strictly immutable**. All fixes to `HumanCore` integration MUST be implemented at the call-sites in `sim.js` / external modules, never by editing `human-core.js`.
- Maintain strict integer satang conservation, physical lot conservation, partition-invariant hourly execution, read-only snapshot purity, and observer-only Thai UI.

Executive Summary Table of Discovered Defects

ID	Category	Severity	Location	Summary of Defect / Latent Time-Bomb
LB-01	Active Logic Bug	**CRITICAL**	`sim.js: recruit()`	Early `return` after checking only `candidates[0]` permanently blocks low/mid-ambition CEOs from

ID	Category	Severity	Location	Summary of Defect / Latent Time-Bomb
				hiring `candidates[1..N]`.
LB-02	Perverse Math Bug	**CRITICAL**	`phone-engineering.js` + `sim.js`	Upgrading display resolution strictly **lowers** `game` score (`153600 / pixels`), increases watts/cost, and `displayQualityScore` is omitted from `consumerUtility()`—making better displays 100% net-negative poison.
LB-03	Clock & Causality Bug	**HIGH**	`sim.js: advance()`	`w.minute -= rem` rewinds world time to the start of the hour before `tick()`, stamping hourly events *earlier* than mid-hour `Edit Hub` commands and delaying non-hour-aligned `eta`s by +60m.

ID	Category	Severity	Location	Summary of Defect / Latent Time-Bomb
LB-04	HumanCore Misuse	**HIGH**	`sim.js: create(), tick()`	All 7 subjects (including `optics`, `sales`, `semiconductor`) are hard-capped by `potential.memory`, leaving 29/30 `HumanCore` aptitudes and `learn()`/`evaluateAbility()`/`workplaceVoice()` unused.
LB-05	Latent Burnout Spiral	**HIGH**	`sim.js: tick()` vs `human-core.js`	`sim.js` uses `'rest'` (`+0.60/h` sleepDebt) instead of `'offDuty'` (`+0.45/h`) for non-work hours; waking `sleepDebt` (`+9.6/d`) barely clears 8h sleep (`-10.8/d`). Any overtime or `<0.89` sleep quality triggers irreversible population-wide burnout.
LB-06	Spec-to-Stock Drift	**HIGH**	`sim.js: spec(), makeDesign()`	`spec()` always pulls from the first array match

ID	Category	Severity	Location	Summary of Defect / Latent Time-Bomb
				(`w.firms.find(...)`) at project start before `acquire()` buys parts—causing assembled phone specs to silently mismatch the actual supplier stock consumed.
LB-07	Physics Saturation & Safety	**MEDIUM-HIGH**	`phone-engineering.js: evaluate(), wafer()`	`photoLowLightScore` saturates at `100` by Level 2–3 due to linear MP multiplication; no upper thermal cut-off (`surfaceC` can exceed `90°C` while `compatible === true`); `wafer()` lacks node-scaled wafer cost/defect curves.
LB-08	Production Loop Flaws	**MEDIUM**	`sim.js: operate()`	Repeat production batches of already-released phones are forced to re-run

ID	Category	Severity	Location	Summary of Defect / Latent Time-Bomb
				`pilot` (`600`) + `release` (`180`); `b.work` and `p.work` desynchronize on mid-hour `pilot -> manufacture` rollover; 82% of brand labor vanishes while waiting on shipment ETA.
LB-09	UI DOM Thrashing Bug	**HIGH**	`ui.js: render(), frame()`	At `1x/5x/20x` speed, `render()` calls `replaceChildren()` on `<select id="firm">` and `Edit Hub` inputs up to 8x/sec, forcibly closing open dropdowns and overwriting user-typed input values.
LB-10	Scale & Audit Time-Bombs	**HIGH (Future)**	`sim.js: transfer(), audit()`	`w.ledger` (`max 1200`) overflows in **3.5 days** due to 255 daily THB-20 meal transfers; `w.offers` grows unboundedly;

ID	Category	Severity	Location	Summary of Defect / Latent Time-Bomb
				`audit()` hardcodes `quantity * 7` parts and `batch === 12`, blocking multi-camera/companion-chip BOMs and variable MOQs.

Detailed Forensic Analysis & Exact Engineering Fixes

LB-01: Recruitment Candidate-0 Blocking Bug (`sim.js: recruit()`)

Exact Code & Failure Proof

In `SPINOFF_SOURCE/js/sim.js`:

```javascript
const candidates = w.people.filter(...).sort((a,b) => b.subjects[need] -
a.subjects[need] || a.id.localeCompare(b.id));
const p = candidates[0]; if (!p) return;
const wage = Math.ceil(p.wage * (1.1 + ceo.profile.personality.ambition / 500));
const accept = wage > p.wage * (p.employerId
  ? (1.12 + p.profile.values.security / 700 + p.profile.personality.patience /
1500 - p.unpaidDays * .04)
  : (1.05 + p.profile.values.security / 2000));
if (!accept) return;
```

- **Why This Is Broken:** Once the initial unemployed pool is hired, `candidates[0]` is globally the single highest-skill employed engineer for `need`. If `candidates[0]` has high `values.security` (e.g., `80`) and `patience` (`75`), their required multiplier is `1.12 + 0.114 + 0.05 = 1.284x`. Any CEO with `ambition < 92` offers `< 1.284x` (`1.1 + 92/500 = 1.284`).

- Because the function executes `if (!accept) return;` instead of iterating through `candidates`, **the firm aborts recruitment completely** without ever evaluating `candidates[1]`, `candidates[2]`, an unemployed specialist at `candidates[5]`, or an unpaid engineer (`unpaidDays = 10`) at `candidates[8]` who would gladly accept!

Required Fix

Iterate over `for (const p of candidates)` (filtered by wage affordability against `firm.cash` runway), compute a utility-to-wage score `(p.subjects[need] / offeredWage)`, and issue the offer to the **first willing and affordable candidate**, not exclusively `candidates[0]`.

LB-02: Display Upgrade "Poison Pill" & Missing Utility Term (`phone-engineering.js` + `sim.js`)

Exact Code & Failure Proof

1. In `phone-engineering.js` (`evaluate()`):

```javascript
game: score((os.installableApps ? compute * 25 * memory : compute * 4)
  * Math.min(1, Math.max(0, ram.capacityMB - os.minRamMB) / 128)
  * Math.min(1, 153600 / Math.max(1, display.pixels)))
```

   Here `153600 = 320 * 480`. Moving to a sharper `480x800` (`384,000` px) or `720x1280` (`921,600` px) display multiplies `taskScores.game` by **`0.40x` or `0.16x`**!

2. Meanwhile, `evaluate()` computes `displayQualityScore`, **but `sim.js` (`consumerUtility()`) never reads `displayQualityScore`**:

```javascript
const features = [
  e.taskScores.call / 100,
  Math.min(1, e.batteryHours / 24),
  Math.min(1, e.photoLowLightScore / 100),
  e.taskScores.web / 200 + e.taskScores.game / 200
];
```

- **Why This Makes Zero Sense:** Choosing a higher-resolution, brighter display **(1)** increases `display.watts` (reducing `batteryHours`), **(2)** increases BOM cost (raising

`d.price`), **(3)** slashes `taskScores.game` via pixel fill-rate penalty, and **(4)** adds **0.000** to visual/media/photo utility! A rational optimizer would put a tiny `128x160` feature-phone screen on a flagship SoC to "cheat" 100 FPS gaming and 30-hour battery life!

Required Fix

1. Include `displayVisualScore` (sharpness PPI, brightness nits, contrast, refresh rate) explicitly in `consumerUtility()` (especially for photography, media, and flagship cohorts).
2. In `taskScores.game`, model GPU fill-rate realism properly: compare `chip.gpuGflops` against `display.pixels * (display.refreshHz / 60)` (`fillRateRatio = gpuGflops / requiredGflopsForResolution`) rather than penalizing `compute` unconditionally against `153,600` pixels, and reward visual immersion when `display.pixels` and `refreshHz` are adequately driven by the GPU.

LB-03: Sub-Hour Clock Rewind & Non-Monotonic Timestamp Bug (`sim.js: advance()`)

Exact Code & Failure Proof

```JavaScript
function advance(w, minutes) {
  const target = w.minute + minutes;
  while (Math.floor(w.minute / 60) < Math.floor(target / 60)) {
    const rem = w.minute % 60;
    w.minute -= rem;
    tick(w);
  }
  w.minute = target;
  return w;
}
```

- **Scenario 1 (Causality Inversion in `events` & `ledger`):** During `1x` playback (`advance(w, 5)`), `w.minute` steps `0 -> 5 -> ... -> 30`. At `00:30` (`w.minute = 30`), the observer pauses and injects cash via `Edit Hub` (`command()`), recording `{ minute: 30 }` in `w.events`. When playback reaches `55 -> 60`, `w.minute -= 55` rewinds `w.minute` to **`0`**, and `tick()` logs events/transfers stamped **`minute: 0` AFTER `minute: 30`**!
- **Scenario 2 (+60m Shipment/Offer Delay):** Inside `tick()`, `w.minute` equals the *start* of the elapsed hour (`H * 60`) rather than the *completed* hour boundary (`(H + 1) * 60`). Any shipment or offer created at a non-zero minute offset (e.g., `w.minute = 30 +`

`leadMinutes`) has `eta = (H + 1) * 60 - 30`, which fails `s.eta <= H * 60` when crossing `(H + 1) * 60`, arriving an hour late!

Required Fix

Advance `w.minute` **to the completed hour boundary** `(Math.floor(w.minute / 60) + 1) * 60` (or pass `boundaryMinute` into `tick()`) consistently:

- Note that `daily(w)` currently checks `hour === 0` (`w.minute % 1440 === 0`). Keep exact schedule semantics clear and consistent: if `tick()` settles the hour `[h*60, (h+1)*60)` at `boundary = (h+1)*60`, derive `scheduleHour = h` for `daily()`/activity selection while stamping completed-hour deliveries (`s.eta <= boundary`), ledger transfers, and events at `boundary` so timestamps never move backward relative to mid-hour commands! Verify with a dedicated sub-hour command + shipment ETA test.

LB-04: `potential.memory` Bottleneck & 85% Unused `HumanCore` Capabilities (`sim.js` vs `human-core.js`)

Exact Code & Failure Proof

`human-core.js` provides 30 distinct `APTITUDES`, 18 `VALUES`, `createSkill()`, `learn()`, `decaySkill()`, `evaluateAbility()`, `deriveAffect()`, `workplaceVoice()`, `deriveAspirations()`, and `evaluateInteraction()`. Yet `sim.js`:

1. Hard-caps **every** subject (`electronics`, `software`, `quality`, `semiconductor`, `optics`, `manufacturing`, `sales`) by `p.profile.potential.memory` (`sim.js` lines 30 & 85)! An optics genius with `visualAcuity: 94, colorDiscrimination: 92, analytical: 88` but `potential.memory: 46` is permanently stuck at `<= 46` in `optics`!
2. Never calls `H.evaluateAbility()` (which already computes `speedMultiplier` and `qualityChance` from `precision`, `sensory`, `composure`, `burnout`, `sleepDebt`, `hunger`, `morale`, and `toolMultiplier`!).
3. Never calls `H.workplaceVoice()` or `H.deriveAffect()` (which already compute `retentionRisk` and `considerLeaving` from `unpaidDays`, `burnout`, `morale`, `wageFairness`, and `stagnantGrowth`!).
4. Never calls `H.deriveAspirations()` (which already computes `savingPropensity [0.08..0.85]`—the exact parameter needed for circular household spending vs. personal savings!).

Required Fix (Zero Edits to `human-core.js`)

Define canonical `SUBJECT_SPECS` in an external module mapping each engineering/commercial subject to proper `HumanCore` aptitude weights (e.g., `semiconductor: { analytical: 1, numerical: 0.9, problemSensitivity: 0.8, memory: 0.5 }`, `optics: { visualAcuity: 0.9, colorDiscrimination: 0.9, analytical: 0.8, precision: 0.8 }`, `sales: { communication: 1, verbal: 0.9, emotionalInsight: 0.8 }`), and wire `H.skillCap(profile, spec)`, `H.evaluateAbility()`, `H.workplaceVoice()`, and `H.deriveAspirations()` directly into `sim.js`!

LB-05: Latent Population-Wide Burnout Spiral (`sleepDebt` Balance in `sim.js`)

Exact Code & Failure Proof

In `human-core.js` (`tickState`):

- Non-sleep hours add `sleepDebt += h * (activity === 'offDuty' ? 0.45 : 0.60)`.
- Sleep hours subtract `sleepDebt -= h * 1.35 * quality`. In `sim.js`, the 24h cycle is `8h 'sleep'` (`-10.80`) and `16h` non-sleep (`'meal'`, `'commute'`, `'work'`, `'rest'`)—never `'offDuty'`—adding `16 * 0.60 = +9.60/day`.
- Margin of safety is only `1.20/day` under ideal `sleepQuality = 1.0`.
- **Latent Failure:** The instant overtime (even 1.5h/day), illness, or `homeComfort < 0.89` is introduced, daily `sleepDebt` delta turns strictly positive (`> 0`), driving every employee in the world past `sleepDebt > 65` -> monotonic `burnout` -> `energy`/`morale` collapse -> permanent work paralysis (`energy < 22 || stress > 80`).

Required Fix

1. Map evening leisure hours (`20:00–22:59`) and weekend/rest days to `'offDuty'` (`0.45/h` sleepDebt, plus `workLifeBalance`/`family` fulfillment recovery built into `human-core.js`!) instead of workplace `'rest'`.
2. Add adaptive recovery: if `p.state.sleepDebt > 35` or `p.state.energy < 30`, let the NPC sleep in at `07:00` or go to bed at `22:00` instead of commuting exhausted.

LB-06: Silent Spec-to-Stock Provenance Drift (`spec()` vs. `acquire()`)

Exact Code & Failure Proof

In `sim.js`:

```javascript
function spec(w, kind, level) {
  const supplier = w.firms.find(f => (OUTPUT[f.role] || []).includes(kind) &&
f.capability >= level);
  ...
}
```

`makeDesign()` calls `spec()` at project creation (using the first matching firm in `w.firms`), whereas `acquire()` later buys parts from whichever supplier wins `supplierQuotes()` (often `supplier-*-alt` or a supplier `player`!).

- **Impact:** Even if Supplier B sells cheaper or higher-capability parts to Brand X, Brand X's phone inherits Supplier A's synthesized spec! In `v0.3.0+`, any disconnect between **design-time target BOM** and **lot-consumed physical BOM** invalidates provenance guarantees.

Required Fix

Bind every phone design to **explicit target `componentSkuId`s** during architecture selection; require `prototype` and `pilot`/`manufacture` stages to consume lots matching those exact `componentSkuId`s (or trigger an explicit, re-evaluated **Engineering Change Order / BOM Substitution** check via `evaluate()` if substituting a compatible alternative SKU).

LB-07: Physics Ceiling Saturation, Missing Thermal Cut-Off & Inverted Wafer Economics (`phone-engineering.js`)

Exact Code & Failure Proof

1. **Score Clamping at `100` (`photoLowLightScore`, `taskScores`):** `cameraQuality = megapixels * (0.5 + sensorAreaMm2 / 10) * processingQuality` grows linearly with `megapixels`, hitting `photoLowLightScore === 100` by Level 2–3 (`16 MP`). All higher-tier cameras tie at `100.0`, blinding `consumerUtility()` to further optical R&D.

2. **No Thermal Safety Cut-Off (`surfaceC`):** `evaluate()` sets `compatible = !reasonsTh.length`, checking power delivery (`battery.maxDischargeW < peakPowerW`) **but never failing `compatible` when `surfaceC > 52°C`**! A phone reaching `85°C` skin temperature passes `validation` as `compatible: true`!
3. **Wafer Cost Constant Across Nodes (`wafer()`):** `wafer()` defaults `waferCostSatang = 3,000,000` across all nodes while `synthesizeChip()` shrinks `dieAreaMm2` with `(nodeNm / 130) ** 1.5`. Shrinking to `22nm` yields ~1,500+ dies per wafer at flat wafer cost, making bleeding-edge chips **10x cheaper per die** than mature `130nm` chips even at low volume!

Required Fix

1. Use diminishing-returns log/Hill curves (or un-clamped internal engineering indices mapped via relative market normalization) instead of linear formulas hard-clamped at `100`, and separate `pixelPitch` low-light physics from daylight megapixel resolution.
2. Add hard thermal qualification failure in `evaluate()`: if steady-state `surfaceC > 50°C` (even after DVFS throttling floor) or junction temp exceeds silicon limits, push `'ความร้อนสะสมเกินเกณฑ์ความปลอดภัยตัวเครื่อง'` to `reasonsTh` (`compatible = false`).
3. Scale `waferCostSatang` and initial `defectsPerCm2` inversely with `nodeNm` (`waferCost ~ (130 / nodeNm)^0.9`), so advanced nodes require high volume and process maturity (D_0 learning) to beat mature nodes on unit cost.

LB-08: Repeat-Batch Re-Piloting, Mid-Hour `WIP` Desync & Idle Engineering Waste (`sim.js: operate()`)

Exact Code & Failure Proof

1. **Repeat Batch Re-Piloting:** When `current.level === level && current.targetSegment === f.strategy.segmentId`, `operate()` starts a reorder at `stage: 'procurement'`, which then runs **full `pilot` (`600` work) and `release` (`180` work) again** on an already-released model!
2. **Work Counter Desync:** On the hour tick where `pilot` (`p.required = 600`) completes with `p.work = 640`, `p.work` resets to `0` (`40` excess work discarded), whereas `b.work` in `w.WIP` retains `640`!
3. **82% Idle Labor Waste During Transit:** When `p.stage === 'prototype'` (waiting on `acquire()`) or `p.stage === 'procurement'` (waiting on part shipments), `operate()` returns early without using `budget`—destroying 82% of paid engineering hours that hour.

Required Fix

1. Route repeat mass-production batches of an already-released SKU Revision directly from `procurement` -> `manufacture` -> `inventory` (skipping redundant `pilot` and `release` ceremonies unless BOM revision changed).
2. Carry over excess work (`const overflow = p.work - p.required`) cleanly across contiguous internal stage transitions or store progress in a single source of truth (`b.work` for physical batches, `p.work` for R&D stages).
3. Redirect unspent engineering `budget` during supply-wait ticks into active background R&D, yield improvement, QA testbench prep, or OTA firmware work.

LB-09: Real-Time UI DOM Destruction Bug (`ui.js: render()` & `frame()`)

Exact Code & Failure Proof

In `SPINOFF_SOURCE/js/ui.js`:

- At `5x` or `20x` speed, `frame()` calls `G.advance(world, n); render();` up to **8–12 times per second**.
- Every `render()` call executes `$('firm').replaceChildren()`, `$('tabs').replaceChildren()`, and `$('content').replaceChildren()`, **destroying and recreating every `<select>`, `<input>`, button, and scroll container in the DOM**!
- **Active Bugs Caused:**
    1. Clicking `<select id="firm">` to switch observed companies while time is running (`1x`/`5x`/`20x`) immediately closes/resets the native dropdown before the user can pick an option!
    2. If the user enters `Edit Hub` and clicks `1x`/`5x` to watch live while editing, `render()` wipes keyboard focus and overwrites `<input>` values on every frame!
    3. Calling `G.snapshot(world)` (`JSON.parse(JSON.stringify(world))`) 8x/sec on the UI thread will stutter mobile browsers once `v0.3+` state grows.

Required Fix

1. **Preserve Stateful Controls:** Update `<select id="firm">` options in place (diffing by `firm.id` length/order) rather than `replaceChildren()` on every tick; never overwrite focused `<input>` / `<select>` elements (`document.activeElement`).
2. **Decouple Simulation Step Rate from UI Render Rate:** Throttle full panel `render()` during fast-forward (`5x`/`20x`) to `~4–5 Hz` (200–250ms) while keeping canvas animation

smooth, and project lightweight read-only view slices rather than deep-cloning immutable historical arrays unnecessarily (while keeping exported `SiliconSim.snapshot(world)` deep-cloned per API contract).

LB-10: Save-Bloat, Ledger Amnesia & Hardcoded `audit()` Constraints (`sim.js`)

Exact Code & Failure Proof

1. **3.5-Day Ledger Amnesia (`w.ledger.length > 1200`):** ~85 people × 3 meals/day (`2,000` satang each) + 85 daily wage transfers = **~340 transfers/day**. `w.ledger` shifts out entries after `1,200` rows = **3.5 simulated days**! Meal micro-transactions wipe out corporate B2B purchase, NRE, and retail sales history almost immediately.
2. **Unbounded `w.offers` Array:** Completed job offers (`status: 'joined'`) remain in `w.offers` forever, bloating save JSONs over multi-year runs.
3. **Rigid `audit()` Magic Constants:** `audit()` currently hard-fails if:
   - `b.inputs !== b.quantity * 7` (forbids 8-part phones with companion ISP/PMIC chips or dual cameras),
   - `b.quantity !== 12` or `b.required !== 240` / `1800` (forbids variable batch sizes, MOQs, or process complexity),
   - `s.key` does not match `/^(chip|ram|storage|display|camera|battery|enclosure):[0-3]$/` (must be updated cleanly in `v0.3+` to validate explicit `skuId` / `lotId` registry references instead of regex levels).
4. **IEEE-754 Float Threshold Drift:** Work accumulators (`f.salesWork`, `f.research`, `p.work`, `b.work`) use floating-point addition. Converting internal work accumulators to **integer milli-minutes** (or integer work points) guarantees 100% cross-engine determinism alongside integer satang.

Required Fix

1. Aggregate daily routine personal meal/subsistence transfers per cohort/day (or store corporate accounting entries in a dedicated bounded `firm.financialHistory` ring buffer separate from personal meal micro-transfers), and prune/archive `joined` offers older than `90` days into a compact counter.
2. Refactor `audit(w)` invariants to validate **conservation laws and relational schema contracts** (`sum(lotInputsConsumed) === batch.reservedPartCount`, `batch.quantity >= contract.minBatchSize`, valid `skuId` foreign keys) rather than hardcoded `12` / `7` / `240` magic numbers.
3. Represent work progress and commercial effort internally as safe integers (`workMilliUnits`) to eliminate floating-point boundary drift permanently.

Verification Checklist for `GPT Sol 6.1` & `Gemini 4 Argon`

When patching the defects above, add targeted regression tests in `SPINOFF_SOURCE/test/simulation.test.cjs` and `browser.test.cjs` verifying:

1. **Recruitment Fallback Test (`LB-01`):** When `candidates[0]` rejects an offer due to high wage/security, the firm successfully hires a willing `candidates[k]` (`k > 0`).
2. **Display Monotonicity / Trade-Off Test (`LB-02`):** Pairing an upgraded display with an adequate GPU improves visual/consumer utility for media/photo/flagship cohorts rather than strictly lowering all utility scores.
3. **Sub-Hour Causality & ETA Test (`LB-03`):** Stepping `advance(w, 5)` with a `command()` at `minute = 30` preserves strictly non-decreasing `minute` stamps in `w.events`/`w.ledger` and delivers shipments at their exact hour boundary.
4. **Multi-Aptitude & Off-Duty Sustainability Test (`LB-04`, `LB-05`):** Subject caps reflect domain-relevant `HumanCore` potentials (not solely `memory`), and a 365-day run with occasional overtime does not trigger irreversible population-wide `sleepDebt` burnout.
5. **Live Playback UI Focus Stability Test (`LB-09`):** Running at `5x` speed in Playwright does not destroy `<select id="firm">` state or overwrite an active `<input>` element's `document.activeElement` focus.